

代码生成与寄存器分配完整模拟卷 05（题目与完整解析）

目录

卷面说明	2
A 卷题目	2
B 完整解析	5

卷面说明

考试时间：120 分钟
总分：100 分
说明：本卷独立覆盖指令选择、指令代价、基本块/流图、活跃性、DAG 优化、描述符和 `getReg`。
目标机语义及代价均在题面中给出。

A 卷题目

一、指令选择与目标代码代价（20 分）

目标机有寄存器 R0、R1，指令及代价如下：

LD	R, x	; R=x	代价 2
ST	x, R	; x=R	代价 2
ADD	Rd, Rs	; Rd=Rd+Rs	代价 1
ADDI	R, k	; R=R+k	代价 1
INC	x	; x=x+1	代价 1

- 1. 为三地址语句 $x=y+z$ 生成目标代码并计算总代价。
- 2. 用通用 LD/ADDI/ST 序列实现 $a=a+1$ ，计算代价。
- 3. 再用 INC a 实现同一语句，计算节省的代价。
- 4. 说明同一 IR 为什么可能对应多个目标指令序列。
- 5. 写出代码生成器在指令选择时必须考虑的三个因素。

二、基本块划分、流图与循环（20 分）

给定三地址指令：

- (1) $b = 1$
- (2) $n = 10$
- (3) $d = 1 + n$
- (4) $c = 2$
- (5) $t1 = 4 * a$
- (6) $t2 = 1 + n$
- (7) $c = c + b$
- (8) if $c < n$ goto (6)
- (9) $a = a + b$
- (10) if $a < n$ goto (4)
- (11) $c = a - b$

- 1. 写出首指令判定规则。
- 2. 列出全部首指令。

3. 划分基本块。
4. 写出完整 CFG 边集合。
5. 写出回边及对应自然循环。

三、活跃性与下一次使用（20 分）

基本块：

```
(1) t1 = a - b
(2) t2 = t1 + c
(3) t3 = t2 + d
(4) a = t3
```

块出口处：

```
LiveOut={a,b,c,d}
t1、t2、t3 不活跃且无后续使用
```

1. 写出对语句 $x=y \text{ op } z$ 反向扫描的更新规则。
2. 计算每条语句之前的活跃变量集合。
3. 写出每个源操作数在块内的下一次使用位置。
4. 指出每条语句的结果在何时变为不活跃。
5. 说明这些信息怎样帮助 `getReg` 选择可覆盖寄存器。

四、基本块 DAG、数组写入与死代码（20 分）

给定基本块：

```
(1) x = a[i]
(2) y = a[i]
(3) a[j] = t
(4) z = a[i]
(5) u = b + 1
(6) v = b + 1
(7) w = v * 1
```

已知无法证明 $i \neq j$ ，块出口活跃变量为：

```
LiveOut={x,z,u}
```

1. 指出可以合并的公共子表达式。
2. 说明数组写入 (3) 为什么会杀死此前的 $a[i]$ 结果。
3. 对 (7) 做代数化简。
4. 删除死代码后写出最终基本块。
5. 区分死代码和不可达代码。

五、描述符、getReg 与两寄存器代码生成 (20 分)

目标机只有 R0、R1，采用两地址指令：

```
LD  R, x      ; R=x
ST  x, R      ; x=R
ADD Rd, Rs    ; Rd=Rd+Rs
SUB Rd, Rs    ; Rd=Rd-Rs
```

三地址码：

```
t1 = a - b
t2 = a - c
t3 = t1 + t2
d  = t3 + t2
```

1. 写出寄存器描述符和地址描述符的定义。
2. 给出一组完整目标代码，允许把临时变量溢出到内存。
3. 逐条写出 R0、R1 中保存的值。
4. 指出哪一步发生 spill，哪一步发生 reload。
5. 写出结束时 d 的地址描述符。

B 完整解析

一、指令选择与目标代码代价解析

$x=y+z$:

```
LD  R0, y      ; 2
LD  R1, z      ; 2
ADD R0, R1      ; 1
ST  x, R0       ; 2
```

总代价= $2+2+1+2=7$ 。

$a=a+1$ 的通用序列:

```
LD  R0, a      ; 2
ADDI R0, 1      ; 1
ST  a, R0       ; 2
```

总代价=5。

使用目标机专用指令:

```
INC a          ; 1
```

节省 $5-1=4$ 个代价单位。

同一 IR 有多个实现的原因:

目标机可能同时提供寄存器-寄存器、寄存器-内存、立即数和专用复合指令。
代码生成器可以选择不同寻址方式、不同寄存器和不同求值顺序，只要语义相同即可。

指令选择考虑:

1. 输入 IR 的层次和可见信息。
2. 目标 ISA、寻址方式和指令语义。
3. 目标代码质量，包括指令数、代价、寄存器压力和执行速度。

二、基本块划分、流图与循环解析

首指令规则:

1. 整个指令序列的第一条指令。
2. 任意条件或无条件跳转的目标指令。
3. 紧跟在条件或无条件跳转之后的指令。

首指令：

- (1)：第一条指令。
- (4)：指令 (10) 的跳转目标。
- (6)：指令 (8) 的跳转目标。
- (9)：紧跟在条件跳转 (8) 后。
- (11)：紧跟在条件跳转 (10) 后。

基本块：

- B1=(1)-(3)
- B2=(4)-(5)
- B3=(6)-(8)
- B4=(9)-(10)
- B5=(11)

CFG 边：

- B1->B2
- B2->B3
- B3->B3 ; (8) 条件为真
- B3->B4 ; (8) 条件为假
- B4->B2 ; (10) 条件为真
- B4->B5 ; (10) 条件为假
- B5->Exit

回边与自然循环：

- B3->B3 是自回边，自然循环={B3}。
- B4->B2 是回边，因为 B2 支配 B4；自然循环={B2,B3,B4}。

三、活跃性与下一次使用解析

反向扫描 $x=y \text{ op } z$ ：

- 先记录语句之后 x 、 y 、 z 的当前 `live/next-use` 信息。
- 再把 x 设为不活跃、无下一次使用。
- 最后把 y 、 z 设为活跃，下一次使用位置设为当前语句。

活跃集合：

- 语句 (4) 之后：{a,b,c,d}
- 语句 (4) 之前：{b,c,d,t3}
- 语句 (3) 之前：{b,c,d,t2}
- 语句 (2) 之前：{b,c,d,t1}
- 语句 (1) 之前：{a,b,c,d}

块内下一次使用：

在语句 (1) 前：a、b 下一次用于 (1)。
在语句 (2) 前：t1、c 下一次用于 (2)。
在语句 (3) 前：t2、d 下一次用于 (3)。
在语句 (4) 前：t3 下一次用于 (4)。

结果活跃区间：

t1 在 (1) 产生，在 (2) 最后一次使用后不活跃。
t2 在 (2) 产生，在 (3) 最后一次使用后不活跃。
t3 在 (3) 产生，在 (4) 最后一次使用后不活跃。
语句 (4) 产生的新 a 在块出口活跃。

寄存器选择：

如果寄存器中的值在当前语句后不活跃，getReg 可以直接覆盖该寄存器。
若所有值都活跃，优先选择下一次使用最晚且已有其他副本的值；否则先 ST 到内存再覆盖。

四、基本块 DAG、数组写入与死代码解析

公共子表达式：

(2) 的 a[i] 与 (1) 的 a[i] 相同，二者之间没有修改 a 或 i，因此 y 可复用 (1) 的结果。
(6) 的 b+1 与 (5) 的 b+1 相同，因此 v 可复用 u 的结果。

数组写入：

无法证明 $i \neq j$ ，所以 $a[j]=t$ 可能修改 $a[i]$ 。
因此 (3) 杀死此前关于数组 a 的读取结果；(4) 不能复用 (1)/(2) 的值，必须重新读取。

代数化简：

$w=v*1 \Rightarrow w=v$

由于 y、v、w 在出口不活跃，其最终赋值可删除。优化后的基本块：

(1) $x = a[i]$
(3) $a[j] = t$
(4) $z = a[i]$
(5) $u = b + 1$

区别：

死代码可能被执行，但其结果不影响后续可观察行为。
不可达代码在 CFG 中不存在从入口到达它的路径，根本不会执行。

五、描述符、getReg 与两寄存器代码生成解析

描述符:

寄存器描述符 `RegDesc(R)`: 寄存器 `R` 当前保存哪些变量的最新值。

地址描述符 `AddrDesc(x)`: 变量 `x` 的最新值当前可在哪些寄存器或内存位置找到。

目标代码:

```
LD  R0, a      ; R0=a
LD  R1, b      ; R1=b
SUB  R0, R1     ; R0=t1
ST   t1, R0     ; spill t1

LD  R0, a      ; R0=a
LD  R1, c      ; R1=c
SUB  R0, R1     ; R0=t2

LD  R1, t1     ; reload t1
ADD  R1, R0     ; R1=t3, R0=t2
ADD  R1, R0     ; R1=d, R0=t2
ST   d, R1
```

寄存器内容变化:

```
初始: R0={}, R1={}
LD R0,a: R0={a}
LD R1,b: R0={a}, R1={b}
SUB: R0={t1}, R1={b}
ST t1,R0: R0={t1}, 且内存 t1 也有最新值
LD R0,a: R0={a}, R1={b}
LD R1,c: R0={a}, R1={c}
SUB: R0={t2}, R1={c}
LD R1,t1: R0={t2}, R1={t1}
第一次 ADD: R0={t2}, R1={t3}
第二次 ADD: R0={t2}, R1={d}
ST d,R1: R1={d}, 内存 d 也有最新值
```

spill 是 `ST t1,R0`, reload 是 `LD R1,t1`。

结束时:

```
AddrDesc(d)={R1, 内存位置 d}
```